

第17讲：Softmax – 怎样把 Logits 真正变成概率分布

从 raw score 到 probability: 为 Cross Entropy 和 LLM next-token prediction 铺路

本讲核心问题：神经网络最后输出的一串任意实数，为什么不能直接叫“概率”？Softmax 到底做了哪两件事？

学完以后你应该能做到

解释 logit 是什么，以及为什么 logit 可以是负数、可以大于 1。

从 $[2.0, 1.0, 0.1]$ 手算 $\exp$ 、求和、归一化，得到 Softmax 概率。

理解 Softmax 保留排序，但改变差距的表达方式。

理解为什么稳定实现要先减去最大 logit。

根据 Tensor Shape 判断 Softmax 应该沿哪个 axis / dim 做。

理解 temperature 怎样改变分布尖锐程度，并连接到 LLM decoding。

知道 Softmax 概率不是“模型一定正确的客观置信度”。



和上一讲的连接：第16讲已经学了概率、 $\exp$ 和 $\log$ 。今天把 $\exp$ 用起来；下一讲 Cross Entropy 会再把 $-\log$ 接上。

1. 先回答：什么是 Logit?

在分类模型里，最后一层通常先输出一串“原始分数”。这些分数还没有满足概率规则，我们通常把它们称为 logits。

类别	logit	现在能否直接叫概率?
class 0	2.0	不能
class 1	1.0	不能
class 2	0.1	不能

关键直觉：Logit 更像模型内部的“相对偏好分数”。它可以是 -8、0.3、12.5，没有 0~1 的限制，也不要求总和为 1。

为什么不能直接把 [2.0, 1.0, 0.1] 当概率？因为概率至少要满足：每一项在 0 到 1 之间；对一个互斥分类问题，所有类别概率之和为 1。

你以前在 LLM 课程中见过 vocabulary logits。一个词表可能有几万甚至十几万个 token，模型会为每个候选 token 输出一个 logit。Softmax 的任务，就是把这整条 logit 向量变成一个合法概率分布。

2. Softmax 的原理：先变成正数，再归一化

Softmax 可以拆成两个非常朴素的动作：



必要公式：

$$\text{Softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$$

公式逐项拆解：

符号	含义	在代码中是什么
z_i	第 i 个类别的 logit	<code>logits[i]</code>
$\exp(z_i)$	把该 logit 映射为严格正数	<code>np.exp(logits[i])</code>
$\sum_j \exp(z_j)$	所有类别 exp 后求和	<code>np.sum(np.exp(logits))</code>
最终除法	把所有正数按比例缩放，使总和等于 1	<code>exp_values / exp_values.sum()</code>

不要背公式：Softmax 的本质就是“正值化 + 归一化”。exp

负责保证正数并放大分数差异；除以总和负责让所有结果加起来恰好等于 1。

3. 具体数字例子：把 $[2.0, 1.0, 0.1]$ 手算一遍

步骤 1：原始 logits。

$$z = [2.0, 1.0, 0.1]$$

步骤 2：逐项取指数。

logit	$\exp(\text{logit})$ (约)
2.0	7.389056
1.0	2.718282
0.1	1.105171

步骤 3：求和。

$$7.389056 + 2.718282 + 1.105171 = 11.212509$$

步骤 4：每一项除以总和。

类别	计算	概率
class 0	$7.389056 / 11.212509$	0.659001
class 1	$2.718282 / 11.212509$	0.242433
class 2	$1.105171 / 11.212509$	0.098566

$$\text{检查: } 0.659001 + 0.242433 + 0.098566 = 1.0$$

你应该形成的心智模型：Softmax 没有“创造”新的类别排序：2.0 最大，所以转换后仍是 class 0 概率最大。它只是把相对分数翻译成了一个可归一化解释的分布。

4. 为什么要用 $\exp$?

$\exp(x)=e^x$ 有两个对 Softmax 很方便的性质。

第一, $\exp(x)$ 永远大于 0。即使 logit 是 -100 , $\exp(-100)$ 仍然是一个非常小但非负的数, 因此不会产生负概率。

第二, $\exp$ 是单调递增函数: 如果 $z_1 > z_2$, 那么 $\exp(z_1) > \exp(z_2)$ 。所以类别排序会被保留下来。

第三, 指数会放大差距。 logit 只差 1, 在 $\exp$ 空间里是一个固定的倍率 $e \approx 2.718$ 。

logit	$\exp(\text{logit})$	相对直觉
0	1.000	基准
1	2.718	约为 2.7 倍
2	7.389	约为 7.4 倍
-1	0.368	仍然为正

科研判断: “Softmax 使用 $\exp$ ” 不是因为 $\exp$ 是唯一可能的归一化方法, 而是因为它具有正值、保序、可微、数学性质好等组合优势, 并与 $\log\text{-likelihood}$ / Cross Entropy 形成非常自然的配合。

5. 为什么最后要除以总和?

假设 $\exp$ 后得到三个正数: $[7.389, 2.718, 1.105]$ 。它们虽然都为正, 但总和是 11.212, 不是概率分布。

把每一项除以同一个总和, 就相当于按比例把整组数字压缩到“总量为 1”的尺度:

$$p_i = \text{positive}_i / \sum \text{positive}_j$$

这一步叫 `normalization` (归一化)。注意: 这里说的是“让一组正数和为 1”的概率归一化, 不要和后面 `BatchNorm`、`LayerNorm` 混为一谈。

常见误区: `Normalization` 这个英文词在深度学习里会出现在不同上下文。`Softmax` 的归一化是概率归一化; `LayerNorm`/`BatchNorm` 是对激活统计量进行标准化, 它们不是同一个操作。

6. 一个非常重要的性质：整体平移不改变 Softmax

实验中我们发现：给所有 logits 同时加上同一个常数，Softmax 结果完全不变。

$$\text{Softmax}(z_i + c) = \text{Softmax}(z_i)$$

原因可以直接拆开：

$$\exp(z_i + c) = \exp(z_i) \cdot \exp(c)$$

分子里有 $\exp(c)$ ，分母每一项也都有 $\exp(c)$ ，最后会被约掉。

logits	Softmax 结果
[2.0, 1.0, 0.1]	[0.659001, 0.242433, 0.098566]
[12.0, 11.0, 10.1]	完全相同
[102.0, 101.0, 100.1]	完全相同

为什么这很重要？因为这允许我们在实现时先把最大 logit 减掉，而完全不改变数学结果。这一步正好解决 $\exp$ 大数溢出的问题。

7. 数值稳定版 Softmax: 先减 max

如果 logits=[1000,1001,999], 直接计算 $\exp(1000)$ 已经远超普通浮点数能稳定表示的范围。实验里 NumPy 得到 inf, 进一步除法变成 nan。

稳定写法: 令 $m=\max(z)$, 先计算 $z-m$ 。

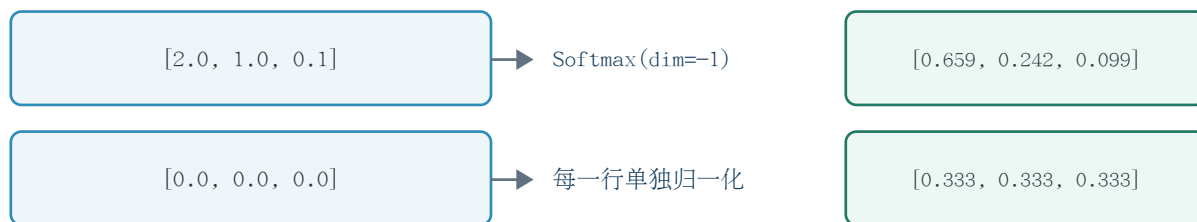
$$\text{Softmax}(z_i) = \exp(z_i-m) / \sum_j \exp(z_j-m)$$

步骤	数值
原 logits	[1000, 1001, 999]
减 max=1001	[-1, 0, -2]
exp	[0.367879, 1.000000, 0.135335]
最终概率	[0.244728, 0.665241, 0.090031]

工程习惯: 以后不要自己写 naive softmax 用于正式训练。NumPy 教学时可以手写稳定版; PyTorch 中直接使用 `torch.softmax / F.softmax`, `CrossEntropyLoss` 里更会进一步采用稳定的 log-softmax 计算。

8. Shape 与 dim: Softmax 到底沿哪个维度做？

对于 Batch 分类问题，logits 可能是 (B,C)：B 个样本，每个样本 C 个类别。Softmax 通常沿类别维 C 做，而不是把整个 Batch 混在一起。



例子：

```
logits.shape = (2,3)
```

softmax(logits, dim=-1) 以后 shape 仍然是 (2,3)，但每一行自己的概率和都是 1。

输入行	输出概率	行和
[2.0,1.0,0.1]	[0.659,0.242,0.099]	1.0
[0,0,0]	[0.333,0.333,0.333]	1.0

Shape 规则： Softmax 通常不改变 Tensor Shape，它改变的是指定 axis 上数字之间的关系。最重要的问题不是“Softmax 后几维”，而是“你究竟在哪个维度上让数字竞争并归一化”。

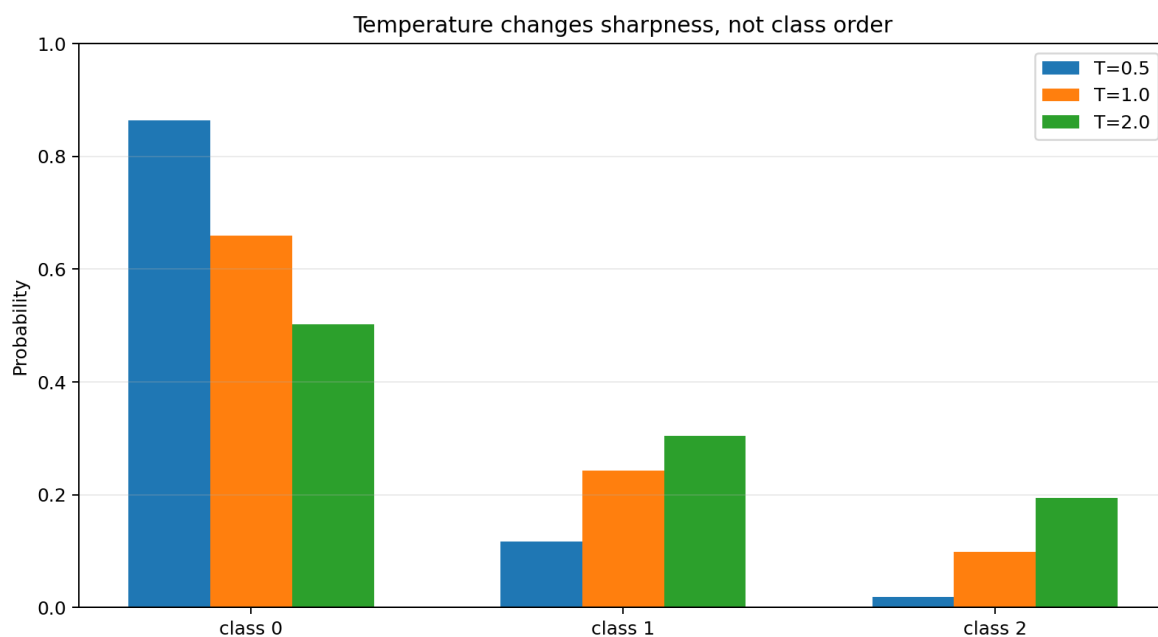
9. Temperature: 同一组 logits 可以变得更尖或更平

Temperature 的常见形式是:

$$p_i = \text{Softmax}(z_i / T)$$

当 $T < 1$, logit 差距被放大, 分布更尖; 当 $T > 1$, 差距被缩小, 分布更平。

T	$\text{Softmax}([2, 1, 0.1]/T)$	最大概率	直觉
0.5	[0.864, 0.117, 0.019]	0.864	更确定、更尖
1.0	[0.659, 0.242, 0.099]	0.659	原始分布
2.0	[0.502, 0.304, 0.194]	0.502	更平、更分散



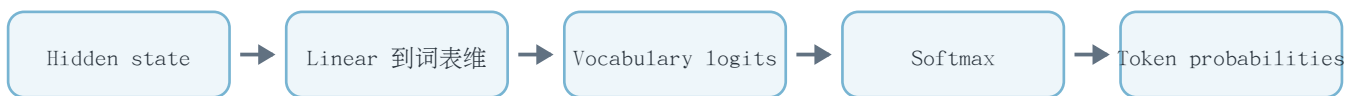
与 LLM decoding 的关系: 生成文本时常会对 logits 做 temperature 调整后再采样。T

小通常更集中于高分 token; T

大通常增加分布多样性。它通常是推理阶段的控制参数, 不等于模型“学会了更随机”。

10. LLM 中 Softmax 到底放在哪里？

你以前在 LLM 课程见过 “Transformer 输出 vocabulary logits”。现在可以从底层重新理解这一段：



如果词表大小 $V=50,000$ ，那么某个位置的最后隐藏向量经过输出线性层后，可以得到 50,000 个 logits。Softmax 沿 vocabulary 维归一化，得到 50,000 个候选 token 的概率。

token	logit	Softmax 概率
猫	1.20	0.193357
狗	2.10	0.475581
AI	-0.30	0.043144
你好	0.80	0.129611
世界	1.00	0.158307

在这个微型词表里，“狗”的 logit

最大，因此概率也最大。但模型最终是否一定输出“狗”，还取决于解码策略：greedy decoding 会选最大值，sampling 则按概率随机抽样。

11. Softmax 还会出现在 Attention 里

Softmax 不只用于最终 token 分类。你以前在 Attention 公式里也见过：

$$\text{Attention}(Q,K,V) = \text{Softmax}(QK^T / \sqrt{d_k}) V$$

这里 Softmax 的对象不是 vocabulary logits，而是某个 Query 对多个 Key 的 attention scores。作用仍然类似：把一组竞争分数变成总和为 1 的权重。



同一个算子，不同语义：输出层 Softmax 的维度通常是“候选类别/token”；Attention 中 Softmax 的维度通常是“当前 Query 可以关注的 Key 位置”。所以读代码时必须先看 Shape 和 dim。

12. Softmax 概率不等于“客观置信度”

这是科研中非常重要的一点。Softmax 输出当然是数学上的概率分布：非负并且总和为 1。但这并不自动保证“模型说 90% 的样本，现实中真的约有 90% 正确”。

如果模型经常给错误答案非常高的 Softmax 概率，我们会说它可能存在 overconfidence / miscalibration。你以前在 LLM 课程里见过 Calibration，这里可以看到它的底层来源之一：Softmax 只负责归一化模型分数，并不会自动校准模型对现实世界不确定性的估计。

概念	能否由 Softmax 本身保证?
概率非负	能
概率和为 1	能
最大 logit 对应最大概率	能
90% 概率意味着长期真的 90% 正确	不能
模型不会高置信度犯错	不能

与 Agent Systems 的关系： Agent

若把模型概率直接当作可靠性指标来决定是否调用工具、是否升级人工审核，可能产生系统性风险。后面做 Agent evaluation / calibration 时必须区分“Softmax 分布”和“经过验证的可靠性概率”。

13. NumPy: 先自己写稳定版 Softmax

```
import numpy as np

def softmax_stable(logits):
    # 1) 找到最大值, 保持最后一个维度, 方便 broadcasting
    m = np.max(logits, axis=-1, keepdims=True)

    # 2) 整体平移, 不改变 Softmax 数学结果
    shifted = logits - m

    # 3) exp 后所有数字都 > 0
    exp_values = np.exp(shifted)

    # 4) 沿最后一个维度求和, 再归一化
    return exp_values / np.sum(exp_values, axis=-1, keepdims=True)

logits = np.array([2.0, 1.0, 0.1])
print(softmax_stable(logits))
# [0.659001 0.242433 0.098566]
```

这里的 `keepdims=True` 很重要: 它让最大值和求和结果保留维度, 从而可以安全地利用 `broadcasting` 对原 Tensor 做减法/除法。

本阶段原则: 我们仍然先自己写 NumPy, 是为了知道框架替你做了什么。正式工程里不需要每次手写 Softmax。

14. PyTorch: `torch.softmax` 在替你做什么？

```
import torch

logits = torch.tensor([2.0, 1.0, 0.1])
probabilities = torch.softmax(logits, dim=-1)

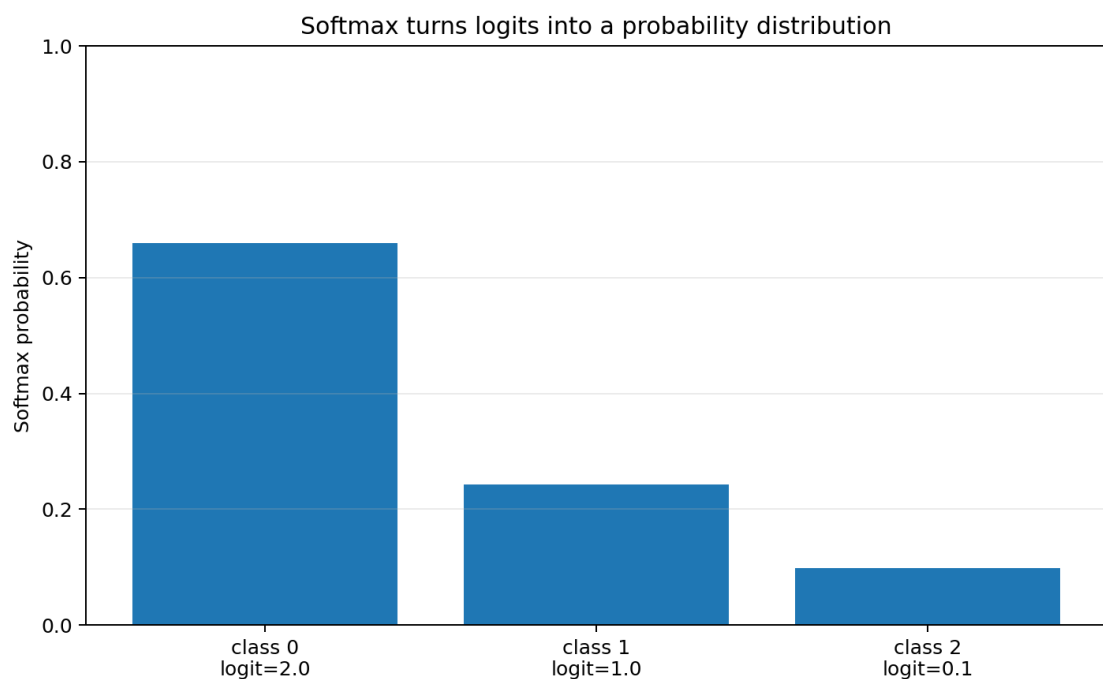
print(probabilities)
# tensor([0.6590, 0.2424, 0.0986])
print(probabilities.sum())
# tensor(1.)
```

本讲实际实验中，NumPy 稳定版和 `torch.softmax` 的最大绝对误差为 0。也就是说，你刚才手写的几步，就是 PyTorch 这个接口的数学核心。

后面学习 Cross Entropy 时会看到一个更重要的工程习惯：训练分类模型时，通常直接把 logits 交给 `CrossEntropyLoss`，而不是先显式做 `softmax` 再传入。原因涉及 `log-softmax` 与数值稳定性，下一讲正式解释。

15. 实验：你应该重点观察什么？

实验	要验证的现象	实际结果
1. 手工 Softmax	$\exp \rightarrow$ 求和 $\rightarrow$ 归一化	$[2, 1, 0.1] \rightarrow [0.659, 0.242, 0.099]$
2. 平移不变	全体 logits + c 不改变结果	最大差异 0
3. 数值稳定	大 logit 直接 $\exp$ 溢出	naive: inf/nan; stable: 正常
4. Temperature	T 改变尖锐程度	T=0.5 最大概率 0.864; T=2 为 0.502
5. Batch dim	每行独立归一化	两行 sum 都为 1
6. LLM 词表	logit $\rightarrow$ token probability	最高概率 token=狗
7. PyTorch 对照	框架与手写一致	最大误差 0



16. 常见误区与本讲最低验收标准

常见误区

误区1: `logit` 就是概率。 - 错; `logit` 是未归一化分数。

误区2: `Softmax` 会改变最大类别。 - 通常不会; `Softmax` 单调保序。

误区3: `Softmax` 后所有概率都会很“确定”。 - 不一定; 相近 `logits` 会得到较平分布。

误区4: `Temperature` 改变类别排序。 - 对正 `T` 而言, 不改变排序, 只改变分布尖锐程度。

误区5: `Softmax` 概率=经过校准的真实正确率。 - 不成立。

误区6: 任何维度都可以随便 `softmax`。 - `dim`

决定“谁和谁竞争”, 语义错误会直接让模型计算失真。

最低验收标准

能手算 3 个 `logits` 的 `Softmax`。

看到 `(B,C)` 或 `(B,T,V)` 能指出通常应该沿 `C / V` 维做分类 `Softmax`。

能解释“减 `max` 不改变 `Softmax`, 却改善数值稳定性”的原因。

能解释 `T<1` 和 `T>1` 分别会怎样改变概率分布。

能说清 `Softmax` 在 LLM 输出层和 `Attention` 中分别归一化什么。

下一讲: 第18讲 `Cross Entropy`。我们会把今天得到的“正确类别概率 `p`”接到 $-\log(p)$, 从而形成真正用于分类和 `next-token prediction` 的训练 `Loss`。